

GCET BLOCKCHAIN CLUB

INNOBLOCK 2.0

Participant handbook

Everything your team needs for the three days: what to set up before you arrive, how the starter kit works, how to build and deploy your dApp, how you're judged, and how to pitch it.

github.com/murthyroshan/innoblock-2.0-starter

DATES

5 – 7 Oct 2026

VENUE

GCET campus

TEAMS

2 – 4 members

NETWORK

Any testnet

Welcome

INNOBLOCK 2.0 is a three-day blockchain hackathon at GCET, organised by the GCET Blockchain Club. Teams of two to four take one problem statement from first sketch to a working prototype on a blockchain testnet, then pitch it to the judges.

To save you a day of boilerplate, every team gets a **starter kit**: a working dApp with a smart contract, a Python backend and a web page, plus step-by-step guides. Clone it, get it running, then turn it into your project.

Starter kit and detailed guides: github.com/murthyroshan/innoblock-2.0-starter

The three days

Day	Date	What happens	Walk out with
Day 1 · Guest lecture	Mon 5 Oct	Guest lecture, problem statements handed out, team ideation	A chosen problem statement, a one-line idea, and your setup finished
Day 2 · Build	Tue 6 Oct	The whole day is yours to build	Contract deployed, app live, full flow tested
Day 3 · Judgment day	Wed 7 Oct	Pitches and live demos before the judges	A tight pitch, a working demo, a submitted repo

Minimum to be judged

- ☐ A smart contract deployed on a **public testnet**, its address in your README and on your first slide
- ☐ At least one transaction from your app visible on the network's **block explorer**
- ☐ A web page that connects a wallet and shows each transaction's status (pending / confirmed / failed)
- ☐ A **public GitHub repo** with a README: setup steps and how to test

AI, a database and a polished UI all earn marks, but only once this minimum works.

Ground rules

- **Testnets only.** Any public testnet is allowed; never mainnet, never real money.
- **Burner wallets only.** Make a fresh wallet for the hackathon. Never use one that has held real funds.
- **AI is allowed**, both as a coding assistant and inside your app, with any provider.
- **The starter kit is optional.** Swap any part (Hardhat, React, a Node backend) if your team prefers. You're judged on what you build.
- **Every member should be able to explain their part.** Judges will ask.

Before you arrive

Do this before Day 2, ideally tonight. Installs and faucet waits are the most common way teams lose their build morning, so split the list across teammates.

1 · Your laptop

- ☐ **Chrome, Brave or Edge** with the **MetaMask** extension from metamask.io
- ☐ In MetaMask: Settings → Advanced → turn on **Show test networks**, then switch the network to **Sepolia**

- ☐ A **new wallet just for the hackathon**. If you already use MetaMask for anything real, add a separate account for this
- ☐ **Python 3.10 or newer** (`python --version`) and **Git** (`git --version`)
- ☐ **VS Code** or any editor

2 • Test tokens

Transactions on a testnet cost free test tokens. Many faucets now ask for a small *real* ETH balance on mainnet, which a new wallet doesn't have. These work without it:

Network	Faucets that work with a new wallet
Ethereum Sepolia (recommended)	QuickNode · Google Cloud (Google account) · PoW faucet (mine in a browser tab)
Base Sepolia	QuickNode · Coinbase Developer Platform
Polygon Amoy	QuickNode · Polygon faucet
Arbitrum Sepolia	QuickNode
OP Sepolia	QuickNode

About **0.05 Sepolia ETH** per wallet is plenty for a whole day. One teammate who gets tokens can send some to the others (MetaMask → Send). If nothing works, ask the organisers on Day 1, not Day 2.

3 • The starter kit, installed

```
git clone https://github.com/murthyroshan/innoblock-2.0-starter.git
cd innoblock-2.0-starter/backend
python -m venv venv
venv\Scripts\activate          # Mac/Linux: source venv/bin/activate
pip install -r requirements.txt
```

If `pip install` works tonight, it will work tomorrow.

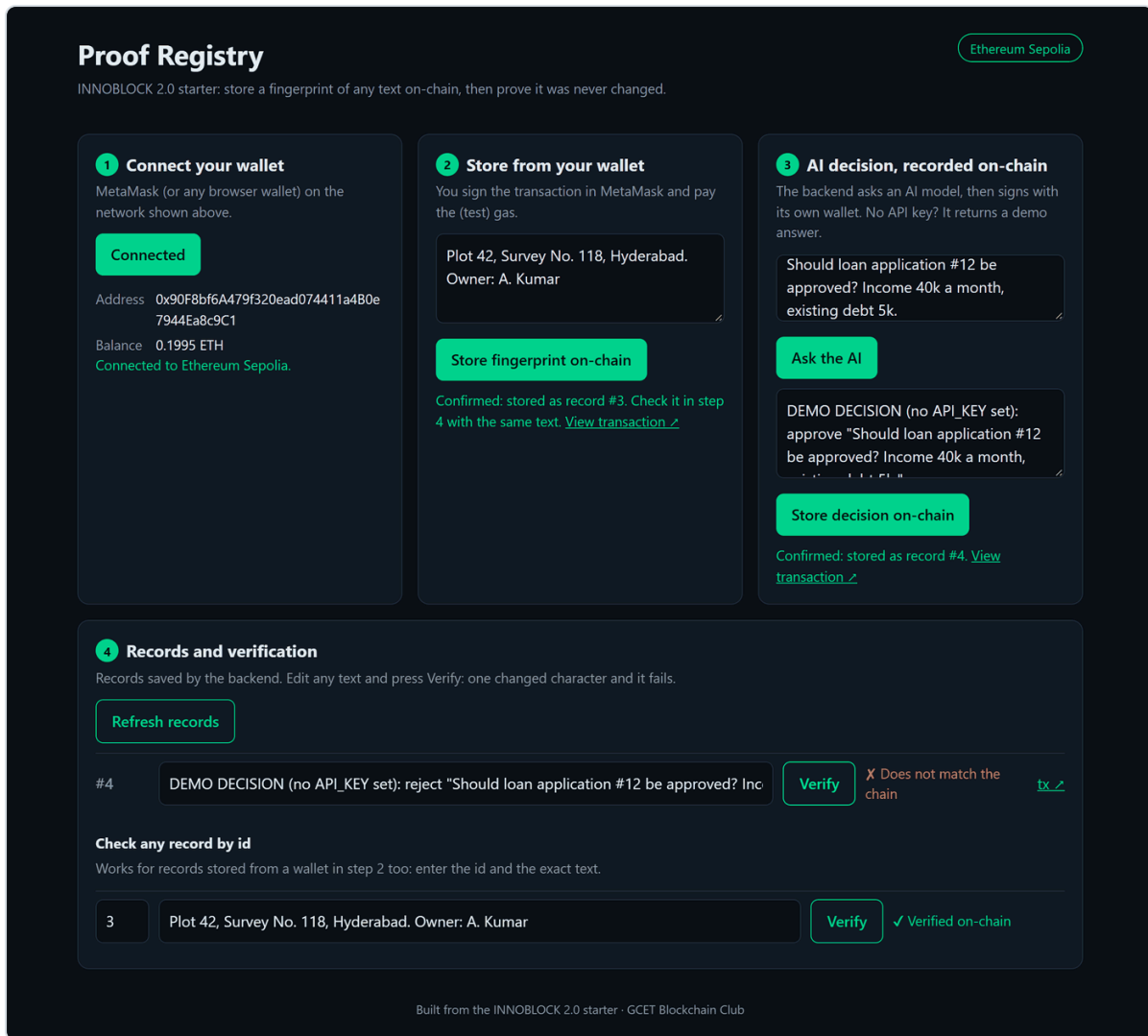
4 • Free accounts (one per team)

[GitHub](#) for your repo · [Render](#) for the backend · [Vercel](#) or [Netlify](#) for the frontend · [Neon](#) or [Supabase](#) for the database · [UptimeRobot](#) to keep the backend awake · an AI provider's API key if your idea uses AI.

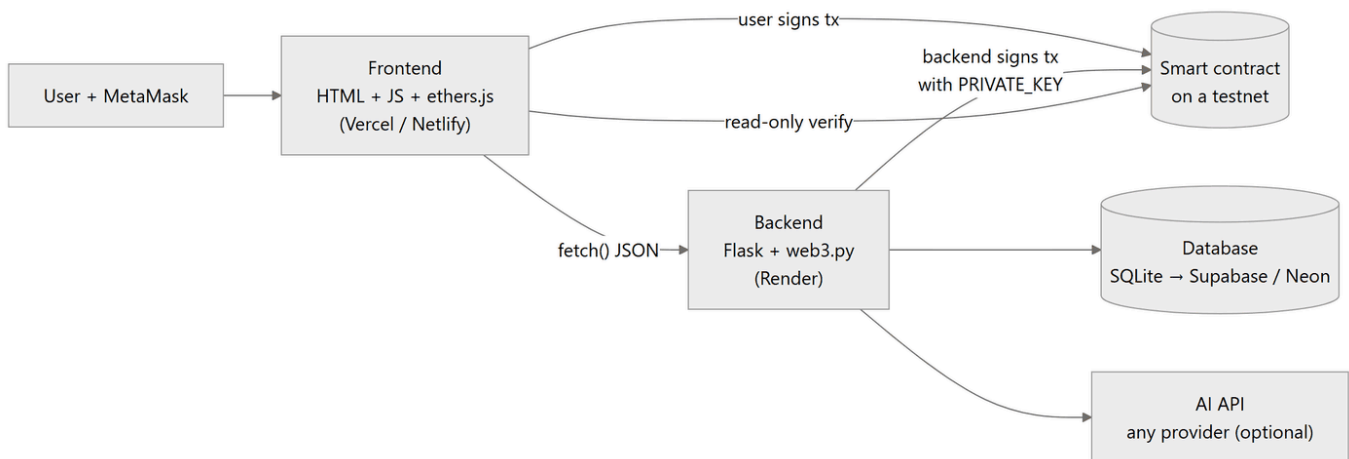
Full checklist and a 10-minute self-test: [Guide 01](#) · [Setup and networks](#).

The starter kit

The demo app is a **proof registry**: it stores a fingerprint (SHA-256 hash) of any text on-chain, then proves later that the text was never changed. It already does what most hackathon dApps need: wallet connection, transactions with live status, an AI step, a database and verification.



How the parts fit together



- **Frontend** (HTML + JavaScript + ethers.js): connects MetaMask, sends transactions the user signs, and verifies records directly against the contract.
- **Backend** (Python Flask + web3.py): does what needs a secret. It calls the AI model, signs transactions with its own wallet, and saves full records in the database.
- **Smart contract** (Solidity): stores one hash per record, emits an event for each one, and answers "does this hash match?" for free.

The core pattern: full record off-chain, fingerprint on-chain

Blockchains are public, permanent and expensive per byte. So keep the **full record** in your database and put only its **hash** (32 bytes) on-chain. Anyone can re-hash the record later and compare. Change one character and the hashes no longer match. The same pattern fits AI decisions, certificates, land records, medical reports and trade signals.

What's in the repo

Path	What it is	You'll change it when
<code>contracts/RecordRegistry.sol</code>	The smart contract	Your idea needs different data or rules
<code>backend/app.py</code>	API: AI call, hashing, signing, database	You add routes or change what's stored
<code>backend/.env</code>	Your secrets and settings (never committed)	Always: create it from <code>.env.example</code>
<code>frontend/config.js</code>	Network, contract address, backend URL	Always
<code>frontend/app.js</code> , <code>index.html</code> , <code>style.css</code>	The web page	You make it your product
<code>docs/</code>	Five detailed guides, each also as a PDF	Whenever you're stuck

Supported networks

The same code works on five EVM testnets; switching is one line in `config.js` plus redeploying the contract.

Ethereum Sepolia is the default and the safest choice.

Network	Chain ID	Currency	Explorer
Ethereum Sepolia	11155111	ETH	sepolia.etherscan.io
Base Sepolia	84532	ETH	sepolia.basescan.org
Polygon Amoy	80002	POL	amoy.polygonscan.com
Arbitrum Sepolia	421614	ETH	sepolia.arbiscan.io
OP Sepolia	11155420	ETH	sepolia-optimism.etherscan.io

Build day, step by step

Get the starter running first, then change it into your idea one piece at a time. Don't start a step until the previous one works.

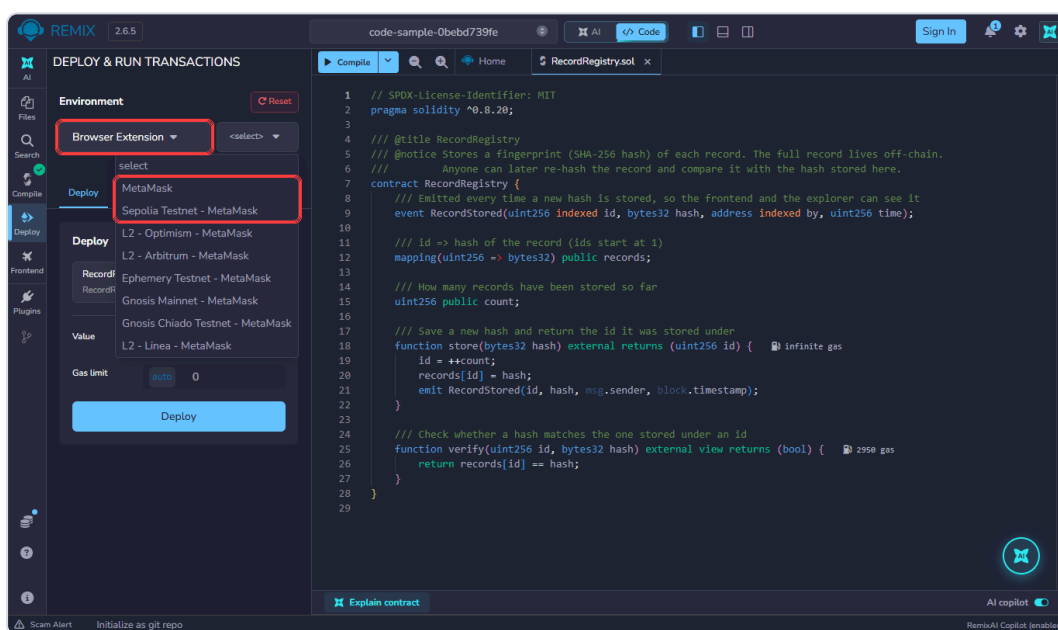
Split the team

Team of	Roles
4	Contract + deployment · backend · frontend · README, slides and pitch
3	Contract + backend · frontend · deployment, README and pitch
2	Contract + backend · frontend + pitch

Agree on the contract's functions early: every redeploy means a new address and ABI everywhere.

1 · Deploy the contract in Remix

Open the contract in Remix with the **Open in Remix** link in the repo's README. In the **Solidity compiler** tab press **Compile**. In **Deploy & run transactions**, set Environment to **Browser Extension**, then **MetaMask**, and press **Deploy**. Copy the contract address.



2 · Verify it on the explorer

On the explorer, open your contract → **Contract** → **Verify and Publish**. Choose *Solidity (Single file)*, the same compiler version as Remix, MIT licence, optimisation off, then paste the source. A green tick means judges can read your code on-chain.

3 · Run the backend

```
cd backend
copy .env.example .env          # Mac/Linux: cp .env.example .env
python app.py
```

Fill in `.env` first: `PRIVATE_KEY` (your burner wallet's key, from MetaMask → Account details), `RPC_URL`, `CONTRACT_ADDRESS` and `EXPLORER_URL` for your network. AI settings are optional. Without a key the app gives demo answers. Check that `localhost:5000/health` says `"ok": true`.

4 · Run the frontend

Set `ACTIVE_NETWORK` and `CONTRACT_ADDRESS` in `frontend/config.js`, then run `python -m http.server 8000` in the `frontend` folder and open `localhost:8000`. Don't double-click the HTML file: wallets don't work on `file://`

pages. Connect, store a record, ask the AI, verify.

5 · Make it yours

Change the contract for your problem statement, copy the new ABI into `backend/abi.json` and `frontend/config.js`, update the address in both, and reshape the page. Set a real AI provider in `.env` if you use AI.

6 · Deploy, then freeze

Put it online (next section). Stop adding features a couple of hours before the build day ends. Then write the README, record a backup demo video and rehearse.

Using AI coding assistants

The repo has two ready-made prompts (build on the starter, or build from scratch) in [Guide 02 · Build](#). Fill in your problem statement and what goes on-chain, paste it, and build one step at a time. If the AI writes something you don't understand, ask it to explain before moving on: judges will ask you.

Deploying for free

What	Where	Key settings
Database	Neon or Supabase	Copy the connection string (Supabase: the <i>Session pooler</i> one) into <code>DATABASE_URL</code>
Backend	Render → Web Service	Root <code>backend</code> · build <code>pip install -r requirements.txt</code> · start <code>gunicorn app:app --timeout 120</code> · add every <code>.env</code> value
Frontend	Vercel (root <code>frontend</code> , preset <i>Other</i>) or Netlify Drop	Put the Render URL in <code>BACKEND_URL</code> first
Keep-alive	UptimeRobot	HTTP(s) monitor on your <code>/health</code> URL every 5 minutes

Then set `FRONTEND_ORIGIN` on Render to your exact frontend URL. Things that catch teams out:

- Free Render services **sleep after 15 minutes** without traffic and take about a minute to wake. UptimeRobot prevents it; open `/health` yourself before you demo.
- Render's disk is **wiped on every restart**, so set `DATABASE_URL` or your records disappear.
- An HTTPS page can't call an `http://` backend. Always use the `https://` Render URL.

Step by step: [Guide 03 · Deploy and security](#).

Security rules

- **Never commit** `.env`. The starter's `.gitignore` excludes it. Run `git status` before every push: if `.env` is listed, stop.
- **No secrets in the frontend**. Everything in `frontend/` is public. Private keys, database URLs and AI keys live only in `backend/.env` and on Render.
- **A leaked key is gone**. Bots drain keys pushed to GitHub within minutes, even testnet ones. Make a new wallet and update `.env` and Render.

- **No personal data on-chain.** The chain is public forever. Store names, health records and ID numbers off-chain, and only their hash on-chain.

Judging

Criterion	Marks	What judges look for
Working prototype & codebase	30	The live demo works end to end on a testnet; transactions visible on the explorer; the repo's code is what runs
Blockchain	25	The chain is needed, not decorative; a sensible on-chain / off-chain split; contract verified on the explorer
Technical quality (GitHub, README, smart contract)	15	A clean public repo with no secrets; a complete README; a readable, commented contract
Pitch and Q&A	15	How well you explain the given problem statement and your solution, on time; every member answers questions about their part
Innovation	10	A fresh angle; AI or other integrations that add real value
UI / UX	5	Easy to follow; clear transaction feedback
Total	100	

The pitch

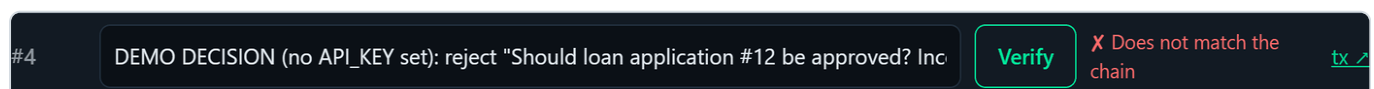
Plan for about **3 minutes plus questions**. The exact slot length is confirmed on Day 1.

Time	Say / show
0:00 – 0:20	Hook: your problem statement in one sentence, with a real example of who it hurts
0:20 – 0:50	Solution: what you built, and why it needs a blockchain
0:50 – 2:20	Live demo: one happy path, ending on the explorer page
2:20 – 2:45	How it works: one architecture slide
2:45 – 3:00	What's next, and the team

Slides: five or six. Title (with your contract address), problem, solution, architecture, demo, what's next.

Demo tips: start with the wallet already connected and funded and the backend awake. Zoom the browser to 125–150%. Paste demo inputs from a notes file. If the live demo breaks, switch to your backup video within 15 seconds.

A strong demo moment: change one character in a stored record and press Verify. It fails, which proves the chain is doing real work.



Questions to prepare for: Why a blockchain and not a database? What exactly is on-chain? What does a transaction cost, and who pays? What if your backend goes down? How do you stop fake data being written? What changes for mainnet?

Checklists

Demo day

- ☐ Frontend deployed; it opens on a phone over mobile data
- ☐ Backend awake: `/health` opened just before you present
- ☐ Database connected: a record created this morning still loads
- ☐ Environment variables set on Render and Vercel, not only on your laptop
- ☐ Full flow tested 30 minutes before your slot
- ☐ Demo wallet **and** backend wallet both hold test tokens
- ☐ Only one wallet extension enabled in the browser
- ☐ Tabs open: the app, the contract on the explorer, a past transaction, the slides
- ☐ Backup demo video (1–2 minutes) saved offline, plus a local copy of the app running
- ☐ Phone hotspot ready; laptop charged; charger and HDMI / USB-C adapter in hand

Submission

- ☐ Public GitHub repo, with no `.env` anywhere in its history
- ☐ README: overview, why blockchain, architecture, setup, environment variables, how to test, team
- ☐ Contract address with a link to its **verified** explorer page
- ☐ Live frontend URL and backend URL
- ☐ Demo video link and slides as PDF
- ☐ Team name, members and problem statement

Where and when to submit is announced on Day 1.

When something breaks

You see	Usually means	Fix
"No wallet found"	No MetaMask, a <code>file://</code> page, or a phone browser	Install MetaMask; serve with <code>python -m http.server</code> ; on a phone use the MetaMask app's browser
"Your wallet has no test ETH"	The wallet is empty on this network	Use a faucet from <i>Before you arrive</i>
"No contract found at CONTRACT_ADDRESS"	Wrong address or network, or an old ABI after a redeploy	Copy the new address and ABI everywhere
"Can't reach the backend"	Backend not running, wrong <code>BACKEND_URL</code> , or Render asleep	Start it, fix <code>config.js</code> , or open <code>/health</code> and wait a minute
CORS error in the console	<code>FRONTEND_ORIGIN</code> doesn't match, or the backend crashed	Match the URL exactly; read Render's logs
<code>nonce too low</code>	MetaMask's history is out of sync	MetaMask → Settings → Advanced → Clear activity tab data
"invalid opcode" when deploying on an L2	Remix targets a newer EVM	Compiler → Advanced Configurations → EVM version <code>cancun</code>
Records vanish on Render	<code>DATABASE_URL</code> not set	Add it in Render's environment

Every error we've seen, with fixes: [Guide 04 · Troubleshooting](#).

Links

Guide (Markdown and PDF in the repo)	Read it when
01 · Setup and networks	Tonight: laptop, wallet, accounts, test tokens, switching networks
02 · Build	Day 2: how the starter works, step by step with screenshots, AI prompts
03 · Deploy and security	Putting it online for free and keeping keys safe
04 · Troubleshooting	Something broke
05 · Pitch and judging	Day 3 prep

Remix: remix.ethereum.org · Sepolia explorer: sepolia.etherscan.io · Announcements: [@gcet_blockchain](#) on Instagram